

Welcome To Linux For You

User Name

Password:

Figure 1: Simple login page

```
<?php echo form_open('login/check'); ?>
<table border='0'>
  <tr align='center'>
    <td colspan='2'><b>Welcome To Linux For You</b></td>
  </tr>
  <tr>
    <td>User Name: </td>
    <td><input type='text' name='username' id='username' /></td>
  </tr>
  <tr>
    <td>Password: </td>
    <td><input type='password' name='password' id='password'
  /></td>
  </tr>
  <tr>
    <td colspan='2' align='center' style='padding-left:40px'><?php
echo form_submit('submit','Submit') ?>
    <input type='button' value=Cancel onClick='window.
location.reload();'/></td>
  </tr>
</table>
<?php echo form_close();?>
<?php echo validation_errors(); ?>
</body>
</html>
```

The above code, when simply loaded in the controller file, will give a browser output somewhat like what's shown in Figure 1. Here's the explanation:

1. `form_open('functionname')` will create a form with the POST method by default. On submission, it will send the form data to the specified function (here, the `check` function of the controller file).
2. `form_submit('submit','Submit')` will create and perform the `submit` button action. Arguments passed are 'type' and 'value'.
3. `form_close()` marks the end of the form.
4. `validation_errors()` will print all validation errors set in the controller file.
5. On cancellation, it will simply reload the page.

File login.php

```
<?php
class Login extends CI_Controller {
function index()
{
$this->load->model('loginmodel');
$this->load->view('loginview');
}
function check()
{
$this->load->library('form_validation'); #1
$this->form_validation->set_rules('username','User
Name','trim|required'); #2
$this->form_validation->set_rules('password','Password','tr
im|required');
if($this->form_validation->run()==FALSE) #3
{
$this->load->view('loginview');
}

$this->load->model('loginmodel');
$username=$this->input->post('username'); #4
$password=$this->input->post('password');
$userCount=$this->loginmodel->check($username,$password); #5

if($userCount[0]['count']>0)
{
echo "Login Successful";
}
else if ($userCount[0]['count']==0)
{
echo "Wrong User Name / Password";
}
} // end of function check
} //end of class
?>
```

And the explanation for this is as follows:

1. We loaded the '`form_validation`' library manually, to use the built-in validation functions.
2. We set the rules for our data fields. Arguments passed are the field name, error message and validation rules, respectively; the most important is the last one. The '`required`' rule makes that field compulsory. You can set multiple rules for a single field using a '|' operator between them. For example, to specify that a valid email address is required, use '`required|valid_email`'. Explore more built-in validation rules in the documentation of `form_validation`.
3. In case form validation fails, the page will simply get reloaded.